

# Linux у проєкті Spring + SPIRE: простий конспект

Доступний конспект термінів, зв'язків і практичного використання

## 1. Що тут взагалі відбувається

У проєкті є два Java/Spring сервіси: `orders-service` і `payments-service`. Вони запускаються не напряму на комп'ютері, а в Linux-контейнерах усередині локального Kubernetes-кластера `kind`. Для кожного сервісу SPIRE видає короткоживучу цифрову ідентичність — SVID. Завдяки їй сервіси встановлюють захищене з'єднання mTLS.

Особлива частина роботи — власний Go-плагін `spire-jvm-attestor`. Він на Linux перевіряє реальний Java-процес через файлову систему `/proc`: чи процес не дебажать, чи не передані небезпечні JVM-параметри та чи запущений саме дозволений JAR. Лише після перевірок SPIRE може видати SVID.

```
Linux host
├─ Docker / containerd
│   └─ kind (локальний Kubernetes-кластер)
│       ├── SPIRE Server: правила та реєстрації
│       ├── SPIRE Agent (DaemonSet): перевіряє процеси на вузлах
│       │   └─ jvm-attestor → читає /proc/<PID>/...
│       ├── orders-service (Java контейнер)
│       └─ payments-service (Java контейнер)
│           └─ ↑ SVID через Unix socket
```

## 2. Linux, термінал і дистрибутив

Linux — ядро операційної системи. Воно керує процесами, файлами, пам'яттю, мережею та правами доступу. Дистрибутив — готова ОС навколо ядра: наприклад Ubuntu, Debian або Alpine Linux. У поточному середовищі використовується Alpine, тому пакети ставляться через `apk`, а не `apt`:

```
sudo apk add openjdk17-jdk maven go git-lfs
```

Shell — текстовий інтерфейс до Linux. У проєкті скрипти починаються з `#!/usr/bin/env bash`: це означає «виконувати файл Bash-інтерпретатором». Корисні базові команди:

```
pwd          # де я зараз
ls           # вміст папки
cd spire-jvm-attestor
cat file     # показати файл
mkdir -p results # створити папку разом із батьківськими
```

## 3. Файлова система і шляхи

У Linux усе організоване як одне дерево папок, яке починається з `/`.

| Шлях                                         | Значення в цій роботі                                       |
|----------------------------------------------|-------------------------------------------------------------|
| <code>/app/payments-service.jar</code>       | типовий JAR усередині контейнера сервісу                    |
| <code>/proc</code>                           | спеціальна файлова система з даними ядра про процеси        |
| <code>/run/spire/sockets/agent.sock</code>   | Unix socket, через який workload звертається до SPIRE Agent |
| <code>/opt/spire/plugins/jvm-attestor</code> | скомпільований плагін, який виконує SPIRE Agent             |
| <code>/tmp</code>                            | тимчасові файли, зокрема в тестових сценаріях               |

Відносний шлях залежить від поточної папки (`./run-all.sh`). Абсолютний шлях починається з `/` і завжди вказує на те саме місце (`/proc/123/status`).

## 4. Права доступу, користувачі та root

Кожен файл має власника, групу і права: читання (`r`), запис (`w`), виконання (`x`). Команда `chmod +x script.sh` робить скрипт виконуваним. Число `0755` означає: власник може читати/писати/виконувати, інші — читати й виконувати.

```
chmod +x attack-tests-attestor/run-all.sh
install -m 0755 bin/jvm-attestor /opt/spire/plugins/jvm-attestor
```

root — адміністратор Linux. Він потрібен для встановлення пакетів, доступу до деяких даних вузла та керування контейнерами. Це причина, чому **sudo** в проєкті використовують обережно. Плагін захищають правами **0755** або **0700**, щоб звичайний користувач не зміг підмінити виконуваний файл.

## 5. Процеси, PID, сигнали і змінні середовища

Процес — запущена програма. Linux дає кожному процесу число PID. Java-сервіс у контейнері — це процес Java; часто його PID усередині контейнера дорівнює **1**.

```
ps          # список процесів
kill <PID>   # надіслати процесу сигнал завершення
kill -0 <PID> # лише перевірити, що процес існує
```

У Bash **!** означає PID щойно запущеної фонові команди, а **wait <PID>** чекає на її завершення. Скрипти застосовують це для **kubectl port-forward** і прибирання процесів через **trap ... EXIT**.

Змінна середовища — пара **ІМ'Я=значення**, яку процес успадковує під час запуску. У проєкті приклади: **K8S\_NAMESPACE**, **SPIFFE\_ENDPOINT\_SOCKET**, **JAVA\_TOOL\_OPTIONS**, **JDK\_JAVA\_OPTIONS**. Частина JVM-змінних може непомітно додати Java-агент або дебагер, тому attestor читає **/proc/<pid>/environ** і позначає підозрілий стан.

## 6. Bash: як працюють тестові скрипти

Bash-скрипти в **attack-tests-attestor/**, **load-tests-attestor/** і **k6-tests/** автоматизують повторювані дії.

| Конструкція                       | Просте пояснення                                                                |                |
|-----------------------------------|---------------------------------------------------------------------------------|----------------|
| <b>set -euo pipefail</b>          | зупинятися на помилці, не допускати порожніх змінних, ловити помилки в pipeline |                |
| <b>VAR="{VAR:-default}"</b>       | взяти значення <b>VAR</b> , а якщо його немає — <b>default</b>                  |                |
| <b>\$(command)</b>                | підставити текстовий результат команди                                          |                |
| <b>`cmd1 \`</b>                   | <b>cmd2`</b>                                                                    | передати вивід |
| <b>&gt; / 2&gt; / 2&gt;&amp;1</b> | перенаправити stdout, stderr або обидва потоки у файл                           |                |
| <b>grep, awk, jq</b>              | знайти текст, обробити стовпці, прочитати JSON                                  |                |

Приклад з тестів: **kubectl logs ... | grep ...** отримує логи SPIRE Agent і залишає лише рядки, важливі для перевірки.

## 7. **/proc**: вікно в ядро Linux

**/proc** (procfs) — не звичайна папка на диску. Її файли створює ядро «на льоту». Для кожного процесу є папка **/proc/<PID>**. Саме тому плагін довіряє цим даним більше, ніж аргументам, які процес може змінити у своїй пам'яті.

| Об'єкт                           | Що показує                             | Як використовується                           |
|----------------------------------|----------------------------------------|-----------------------------------------------|
| <b>/proc/&lt;pid&gt;/status</b>  | стан процесу, зокрема <b>TracerPid</b> | anti-debug: виявити <b>strace / ptrace</b>    |
| <b>/proc/&lt;pid&gt;/cmdline</b> | аргументи запуску Java                 | знайти небезпечні JVM flags; fallback для JAR |
| <b>/proc/&lt;pid&gt;/environ</b> | змінні середовища                      | знайти небезпечні Java options                |
| <b>/proc/&lt;pid&gt;/fd/</b>     | відкриті файлові дескриптори           | знайти JAR, який реально відкрито процесом    |

| Об'єкт                                    | Що показує                        | Як використовується             |
|-------------------------------------------|-----------------------------------|---------------------------------|
| <code>/proc/&lt;pid&gt;/maps</code>       | відображення файлів у пам'ять     | знайти JAR, що memory-mapped    |
| <code>/proc/&lt;pid&gt;/map_files/</code> | kernel handle до mapped-діапазону | прочитати mapped JAR через ядро |

## Файловий дескриптор, inode і symlink

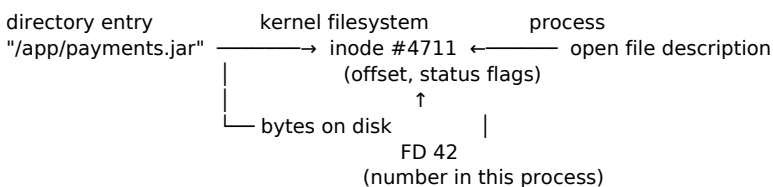
Файловий дескриптор (FD) — маленьке число, яким процес посилається на вже відкритий файл. inode — внутрішній об'єкт файлової системи з метаданими та даними файлу. Назва файлу — це лише посилання на inode. Symlink — «ярлик» на інший шлях.

Це важливо для захисту: якщо Java вже відкрила чистий JAR, `/proc/<pid>/fd/<N>` прив'язаний до його inode. Зловмисник може поміняти шлях або symlink, але це не переключить уже відкритий дескриптор на інший файл. Тому attestor хешує JAR через FD або `map_files`, а не просто за шляхом `/app/payments-service.jar`.

`mmap` означає відображення файлу в пам'ять. Через це JAR може з'явитися у `maps`. Плагін бере об'єднання `maps` у `fd`, а не перше непорожнє джерело: так зайвий JAR не сховається, якщо дозволений JAR відображено в пам'ять.

### 7.1. Повна модель файла: ім'я → inode → open file description → FD

Ці чотири поняття описують різні рівні. Назва файла, сам файловий об'єкт, конкретне відкрите ядром посилання та номер у процесі — не одне й те саме.



| Рівень                      | Що це                                                                              | Що змінюється при перейменуванні                                            |
|-----------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Directory entry / pathname  | запис «ім'я → inode» у напці; наприклад, <code>/app/payments.jar</code>            | може змінитися або зникнути                                                 |
| inode                       | об'єкт файлової системи: тип, права, owner, size, timestamps і посилання на дані   | лишається тим самим, поки є link або відкрите посилання                     |
| Open file description (OFD) | об'єкт ядра, створений <code>open()</code> ; тримає поточний offset і status flags | лишається живим, поки процеси тримають FD                                   |
| File descriptor (FD)        | маленьке число у таблиці конкретного процесу: 0, 1, 2, 3...                        | закривається через <code>close()</code> ; номер можна потім перевикористати |

FD — не сам файл, а номер-ключ у таблиці одного процесу. У різних процесів FD 3 може означати різні файли. Два незалежні `open("/app/a.jar")` створюють різні OFD з різними позиціями читання; `dup()` або `fork()` можуть дати кілька FD, які вказують на один OFD і ділять його offset.

### 7.2. `open()`, `close()` і `pread()`

`open(path)` проходить шлях крізь directory entries, знаходить inode, створює OFD і повертає FD. `close(fd)` прибирає FD з таблиці процесу. Якщо це було останнє посилання на OFD, ядро звільняє OFD.

У OFD є поточна позиція у файлі (file offset). `read(fd, ...)` читає з цієї позиції та рухає її вперед. `pread(fd, ..., offset)` читає з явної позиції, але не змінює спільний offset. Це важливо для attestor: хешування не має впливати на стан FD JVM і не має залежати від того, що JVM саме читає.

JVM already opened JAR → FD 42 → OFD(offset=...) → inode clean.jar

attestor opens /proc/<pid>/fd/42 → kernel resolves the target object  
attestor uses positioned reads → hashes bytes without changing JVM's offset

### 7.3. Hard link, symbolic link і unlink

Hard link — це один directory entry на той самий inode. Два різні імені можуть бути тим самим файлом. Symbolic link (symlink) — маленький спеціальний файл, де записано інший шлях; під час `open()` ядро розгортає цей шлях.

```
/app/payments-service.jar --symlink--> /tmp/decoy.jar
```

`unlink(path)` прибирає лише directory entry (назву). Він не стирає байти одразу. Якщо файл досі має відкритий FD або mmap, inode й дані живуть, доки ядро не втратить останнє посилення. У `/proc/<pid>/fd` та `/proc/<pid>/maps` такий файл може мати суфікс `(deleted)`.

Це ключ до `bypass-symlink.sh`: якщо JVM відкрила добрий JAR, а потім хтось підмінив шлях symlink-ом на decoy JAR, kernel handle все одно стосується старого inode. Хешування шляху могло б перевірити decoy; хешування через FD перевіряє файл, з яким JVM реально працює.

### 7.4. rename, replace і TOCTOU

`rename()` може атомарно перемістити або замінити directory entry. Типова гонка TOCTOU (time of check to time of use) має вигляд:

1. Перевірка читає `/app/payments.jar` і бачить чистий файл А.
2. Атакувальник замінює pathname на файл В.
3. Програма запускає або читає інший файл, ніж той, який перевіряли.

Якщо перевірка спочатку отримує kernel handle до вже відкритого файла, потім читає та хешує той самий об'єкт, цей клас помилки суттєво звужується. У

`HashCode.GetOrComputeByPath` файл спочатку відкривають, далі `stat()` роблять на відкритому FD і з того самого FD обчислюють SHA-256: метадані та байти належать одному об'єкту, а не двом різним станам шляху.

### 7.5. /proc/<pid>/fd: таблиця FD

`/proc/<pid>/fd/` містить один запис на кожен відкритий FD процесу. Записи виглядають як symbolic links, але це спеціальні procfs magic links, які ядро створює як погляд на вже відкриті об'єкти іншого процесу.

```
ls -l /proc/<pid>/fd  
readlink /proc/<pid>/fd/3
```

0, 1, 2 за домовленістю означають stdin, stdout і stderr. Інші числа залежать від програми. Вони можуть вести до звичайного файла, pipe, socket (`socket:[inode]`) або анонімного kernel-об'єкта. Плагін бере лише кандидати на JAR, звіряє inode та хешує їх через `/proc/<pid>/fd/<N>`.

Доступ до `/proc/<інший-pid>/...` не є безумовним: Linux перевіряє UID, ptrace-подібні права, namespaces і capabilities. Тому SPIRE Agent у конфігурації має `hostPID: true` та працює як привілейований компонент, який треба захищати.

### 7.6. mmap, /proc/<pid>/maps і map\_files

`mmap()` відображає частину файла у віртуальну пам'ять процесу. Тоді програма звертається до байтів як до пам'яті, а ядро підвантажує потрібні сторінки. Після успішного `mmap()` початковий FD можна закрити — mapping продовжить існувати.

`/proc/<pid>/maps` — текстовий список memory mappings:

```
address-range perms offset device inode pathname  
7f...7f... r--p 0000 08:01 4711 /app/payments-service.jar
```

- `address-range` — діапазон віртуальних адрес;

- **perms** — права mapping (**r**, **w**, **x**, **p** private або **s** shared);
- **offset** — позиція у файлі, з якої починається mapping;
- **device** та **inode** — ідентичність файла;
- **pathname** — назва, якщо ядро може її показати.

`/proc/<pid>/map_files/` дає записи за діапазоном адрес, які ведуть до mapped file. Це допомагає отримати kernel handle до файла, який досі відображено у пам'яті. Читання таких посилань може вимагати додаткових capabilities.

У Spring Boot fat JAR може читатися через `pread()` і бути видимим лише у FD table, а інший JAR може бути `mmap`-нутий і бути видимим у `maps`. Тому `jvm-atteator` об'єднує обидва джерела:

```
candidates = JARs from /proc/<pid>/fd ∪ JARs from /proc/<pid>/maps
```

Якщо взяти лише перше непорожнє джерело, можна приховати додатковий FD, показавши дозволений mapping. Саме це моделює `bypass-mmap-shadow.sh`.

## 7.7. `stat()`, `device`, `inode`, `size`, `mtime` і `ctime`

`stat()` повертає метадані файла. Для кешу хешів одного `pathname` недостатньо: той самий шлях завтра може вести на інший файл. Ключ `HashCode` містить `path`, `device`, `inode`, `size`, `mtime` і `ctime`.

| Поле                         | Практичний сенс                                                                                   |
|------------------------------|---------------------------------------------------------------------------------------------------|
| <code>device (st_dev)</code> | файлова система/пристрій; <code>inode</code> унікальний лише всередині одного <code>device</code> |
| <code>inode (st_ino)</code>  | конкретний файловий об'єкт у цій файловій системі                                                 |
| <code>size</code>            | швидко ловить типову зміну вмісту                                                                 |
| <code>mtime</code>           | час зміни вмісту файла                                                                            |
| <code>ctime</code>           | час зміни <code>inode</code> -метаданих; не «час створення» на Linux                              |

Можна переписати файл так, щоб `inode` і `size` лишилися, а потім спробувати відновити старий `mtime`. Зміна через `utimensat` все одно рухає `ctime`, тому його присутність у ключі ускладнює застарілий `cache hit`. Це не замінює коректних прав доступу та `kernel handle`, але є ще одним шаром захисту.

## 7.8. Межі гарантій `/proc`

`/proc` дає дані, які ядро формує про реальний процес. У межах обраної `trust boundary` це сильніше джерело, ніж рядок командного запуску, який процес може змінити у своїй пам'яті. Але це не магія:

- читання `maps` може бути гонкою, бо `memory map` процесу здатний змінюватися паралельно;
- доступ до чужого `/proc/<pid>` контролюється Linux permissions/capabilities;
- скомпрометований kernel або привілейований host зламає саму `trust boundary`;
- плагін перевіряє факти, але SPIRE registration entry вирішує, які facts прийнятні.

Тому дизайн проекту поєднує `kernel facts (/proc)`, криптографічні хеші (SHA-256), `policy selectors` і короткоживучі SVID. Жоден шар окремо не вирішує всю задачу.

## 8. Хеші, цілісність і кеш

SHA-256 — криптографічний хеш: короткий відбиток вмісту файла. Зміна навіть одного байта дає інший хеш. Плагін формує:

- `jar_sha256` — хеш кожного знайденого JAR;
- `jar_set_sha256` — хеш від усього відсортованого набору `path:hash`.

Другий хеш важливий: він виявляє додатковий JAR у classpath, навіть якщо дозволений JAR усе ще є. Для швидкодії `HashCode` кешує хеші. Його ключ містить шлях, device, inode, розмір, `mtime` і `ctime`. `mtime` — коли змінено вміст; `ctime` — коли змінено inode-метадані. Це допомагає не повернути старий хеш, якщо файл переписали, а його `mtime` намагалися відновити.

## 9. Debugging і тестові атаки

`ptrace` — Linux-механізм, який дозволяє одному процесу спостерігати за іншим. `strace` — утиліта, що використовує `ptrace` для трасування системних викликів. Якщо JVM трасується, у `/proc/<pid>/status` поле `TracerPid` не нульове; плагін повертає `debug_clean=false`.

Тести також перевіряють:

- небезпечні JVM прапорці (`-javaagent`, `JDWP`, `JMX`);
- небезпечні змінні оточення;
- JVM Attach API socket `.java_pid...`;
- підміну, додавання або надто великий JAR;
- symlink і `mmap`-спроби обійти перевірку.

Це контрольовані лабораторні тести у kind-кластері. `hostPID: true` у тестовому pod дозволяє бачити PID вузла, але це підвищені права, які не варто давати звичайному застосунку.

## 10. Контейнери: Docker, образ і Dockerfile

Контейнер ізолює процес разом із його бібліотеками та файловою системою. Docker image — шаблон контейнера; Dockerfile — рецепт його створення. У сервісах використано `eclipse-temurin:17-alpine`: Linux Alpine + Java 17.

```
FROM eclipse-temurin:17-alpine
COPY target/payments-service.jar /app/payments-service.jar
ENTRYPOINT ["java", "-jar", "/app/payments-service.jar"]
```

Контейнер не є віртуальною машиною: він ділить ядро Linux із host, але має ізольовані namespaces та ліміти cgroups. Це пояснює, чому `/proc` і доступ до PID потрібно проєктувати обережно.

## 11. containerd, registry і multi-architecture

Kubernetes запускає контейнери через container runtime; у kind вузлах це `containerd`. Образи можна зібрати через `docker build`, завантажити в kind (`kind load docker-image`) або відправити в registry. У Makefile є налаштування локального registry `localhost:5000` і дзеркала для containerd.

`GOOS=linux` під час `go build` означає «зібрати бінарник для Linux». `linux/amd64` і `linux/arm64` — різні архітектури процесора. `docker buildx` створює multi-architecture images.

## 12. Kubernetes і kind

Kubernetes (K8s) керує контейнерами декларативно: YAML описує бажаний стан, а кластер намагається його підтримувати. kind = Kubernetes IN Docker: локальний кластер, де вузли Kubernetes самі є Docker-контейнерами.

| Ресурс                | Роль у цій роботі                                              |
|-----------------------|----------------------------------------------------------------|
| Namespace             | логічна ізоляція; SPIRE живе у <b>spire</b>                    |
| Pod                   | найменша одиниця запуску, містить один або кілька контейнерів  |
| Deployment            | підтримує потрібну кількість pod'ів для <b>orders/payments</b> |
| DaemonSet             | запускає один SPIRE Agent на кожному вузлі                     |
| StatefulSet           | запускає SPIRE Server зі стабільним ім'ям і сховищем           |
| Service               | стабільне DNS-ім'я та доступ до pod'ів                         |
| ConfigMap             | не секретна конфігурація: agent/server config, hash manifest   |
| ServiceAccount + RBAC | ідентичність pod'а та дозволи до Kubernetes API                |
| Volume                | спосіб дати контейнеру файли або дані                          |

**kubectl** — CLI для Kubernetes. Основні приклади:

```
kubectl get pods -n spire
kubectl apply -f payment-service-deployment.yaml
kubectl logs -n spire daemonset/spire-agent -c spire-agent
kubectl exec -n spire <pod> -- sh
kubectl rollout status deployment/payments-service -n spire
kubectl port-forward service/payments-service 8080:8080
```

## 13. Kubernetes-мережа, volumes і probes

Pod'и знаходять сервіс за DNS-іменем Kubernetes Service, наприклад **payments-service**. Це працює поверх TCP/IP. **kubectl port-forward** тимчасово прокидає порт із кластера на твій комп'ютер.

hostPath монтує шлях із Linux-вузла у pod. У цій роботі **/run/spire/sockets** монтується до workload-подів, щоб вони бачили Unix socket SPIRE Agent. emptyDir — тимчасова папка для одного pod; нею initContainer передає бінарник jvm-atteator Agent-контейнеру. PersistentVolumeClaim — постійне сховище; SPIRE Server тримає там дані.

Liveness probe перевіряє «контейнер живий?», а readiness probe — «контейнер готовий приймати трафік?». Kubernetes перезапускає або не маршрутизує трафік відповідно до цих перевірок.

## 14. SPIFFE, SPIRE, SVID і mTLS

SPIFFE — стандарт ідентичностей для workload'ів. SPIRE — реалізація SPIFFE.

- SPIRE Server зберігає правила: кому яка ідентичність дозволена.
- SPIRE Agent працює на вузлі, перевіряє workload і видає йому SVID.
- SVID — короткоживучий сертифікат X.509 або JWT, наприклад **spiffe://.../service/payments**.
- Selector — факт про workload, наприклад **jvm:debug\_clean=true** або конкретний **jar\_set\_sha256**.
- Registration entry — правило Server: якщо selectors збігаються, видати певний SVID.

У цьому проєкті Agent викликає JVM-плагін, отримує selectors, порівнює їх із registration entry і або видає SVID, або відмовляє. SVID приходить через Unix socket **agent.sock**; він не лежить як звичайний секрет у YAML.

TLS шифрує з'єднання. mTLS означає, що і клієнт, і сервер показують сертифікат. **orders** і **payments** використовують SVID для взаємної перевірки. Старе mTLS-з'єднання може



залишатися відкритим до нового TLS handshake, тому attack-тести часто перезапускають pod і перевіряють саме першу видачу SVID.

## 15. Спостережуваність і навантаження

Логи показують події в тексті; `kubectl logs` читає їх із контейнера. Prometheus періодично збирає метрики (CPU, пам'ять, latency), а Grafana показує їх на dashboard. k6 генерує HTTP-навантаження та вимірює поведінку сервісів під навантаженням.

Тому в репозиторії є `prometheus/`, `k6-tests/`, `load-tests-atteator/` і `attack-tests-atteator/`: вони відповідають на різні запитання — «чи захист спрацював?», «чи система витримала?» і «скільки це коштує за часом/ресурсами?».

## 16. Практичний маршрут роботи

# 1. Переконатися, що базові інструменти доступні

```
java --version
go version
docker version
kubectl version --client
kind version
```

# 2. Зібрати і протестувати Go-плагін

```
cd spire-jvm-atteator
go test ./...
make build
```

# 3. Зібрати Java-сервіси

```
cd ..
mvn -B -ntp clean verify
```

# 4. Підняти/налаштувати локальне середовище через wsldev

```
wsldev up --name kind
wsldev spire deploy --atteator custom-jvm
wsldev app deploy payments orders
```

# 5. Запустити контрольовані security-тести

```
cd attack-tests-atteator
chmod +x *.sh
./run-all.sh
```

## 17. Короткий словник

| Термін             | Одним реченням                                   |
|--------------------|--------------------------------------------------|
| PID                | номер процесу в Linux                            |
| <code>/proc</code> | дані ядра про систему й процеси у вигляді файлів |
| FD                 | посилання процесу на відкритий файл/socket       |
| inode              | реальний файловий об'єкт за назвою файлу         |
| symlink            | ярлик на інший шлях                              |
| hash/SHA-256       | відбиток вмісту для перевірки цілісності         |
| container          | ізолюваний Linux-процес із власним оточенням     |
| image              | шаблон, з якого створюють контейнер              |
| Kubernetes         | система керування контейнерами                   |
| kind               | локальний Kubernetes у Docker                    |
| Pod                | одиниця запуску контейнерів у Kubernetes         |
| DaemonSet          | один pod на кожному вузлі                        |
| Unix socket        | локальний канал обміну даними між процесами      |
| SPIRE              | система видачі ідентичностей workload'ам         |
| SVID               | короткоживуча ідентичність workload'а            |
| mTLS               | TLS, де перевіряються обидві сторони             |

Головна ідея роботи: Linux дає достовірні факти про запущений JVM-процес через `/proc`; `jvm-attestor` перетворює їх у `selectors`; SPIRE за `selectors` вирішує, чи заслуговує `workload` на SVID; SVID дозволяє сервісам безпечно спілкуватися через mTLS у Kubernetes.